

# Tokens definidos

T\_0=1 valor numérico 1 para optimizar el reconocimiento y parsing de cada token sin compararlos con otros símbolos

T\_1=2 valor numérico 2 para optimizar el reconocimiento y parsing de cada token sin compararlos con otros símbolos

T\_2=3 valor numérico 3 para optimizar el reconocimiento y parsing de cada token sin compararlos con otros símbolos

T\_3=4 valor numérico 4 para optimizar el reconocimiento y parsing de cada token sin compararlos con otros símbolos

T\_4=5 valor numérico 5 para optimizar el reconocimiento y parsing de cada token sin compararlos con otros símbolos

T\_5=6 valor numérico 6 para optimizar el reconocimiento y parsing de cada token sin compararlos con otros símbolos

T\_6=7 valor numérico 7 para optimizar el reconocimiento y parsing de cada token sin compararlos con otros símbolos

T\_7=8 valor numérico 8 para optimizar el reconocimiento y parsing de cada token sin compararlos con otros símbolos

T\_8=9 valor numérico 9 para optimizar el reconocimiento y parsing de cada token sin compararlos con otros símbolos

**ID=10:** Este token identifica las **variables** dentro de la gramática. Se define como cualquier secuencia de letras ([a-zA-Z]+). Esto permite que ID represente nombres de variables compuestos de letras, como a, b, x, etc.

**NUMBER=11:** Representa los **números enteros o decimales** en la gramática. ¿Está definido para aceptar secuencias de dígitos con una parte opcional decimal ([0-9]+ ('.' [0-9]+)?). Esto permite valores como 123, 45.67 o 0.5.

**WS=12:** Este token corresponde a los **espacios en blanco** (o White Space). Incluye espacios, tabulaciones y saltos de línea ([\t\r\n]+) y utiliza la instrucción -> skip para que estos caracteres se ignoren durante el análisis léxico, evitando que interfieran en el procesamiento de las expresiones.

**'('=1:** Representa el **paréntesis de apertura**. Este símbolo permite agrupar expresiones y controlar la precedencia de operaciones.

**)'=2:** Representa el **paréntesis de cierre**, utilizado para cerrar grupos de expresiones.

**'-=3:** Define el operador **de resta** o el operador **unario de negación** cuando se usa al inicio de una expresión o término.

**'^'=4:** Es el operador de **potencia**, que permite elevar una expresión a la potencia de otra.

**'\*=5:** Representa el operador de **multiplicación** para multiplicar dos expresiones.

**'/=6:** Define el operador de **división**, utilizado para dividir una expresión por otra.

**'+=7:** Es el operador de **suma**, que permite la adición entre dos expresiones.

**'&&'=8:** Representa el operador lógico **AND**, utilizado en operaciones lógicas para evaluar si ambas expresiones son verdaderas.

**'||'=9:** Define el operador lógico **OR**, utilizado para evaluar si al menos una de las expresiones es verdadera.

|    |           |
|----|-----------|
| 1  | T__0=1    |
| 2  | T__1=2    |
| 3  | T__2=3    |
| 4  | T__3=4    |
| 5  | T__4=5    |
| 6  | T__5=6    |
| 7  | T__6=7    |
| 8  | T__7=8    |
| 9  | T__8=9    |
| 10 | ID=10     |
| 11 | NUMBER=11 |
| 12 | WS=12     |
| 13 | '('=1     |
| 14 | ')'=2     |
| 15 | '-'=3     |
| 16 | '^'=4     |
| 17 | '*'=5     |
| 18 | '/'=6     |
| 19 | '+'=7     |
| 20 | '&&'=8    |
| 21 | '  '=9    |

# Diseño de la gramática

La gramática **Examen** está estructurada para interpretar y procesar expresiones aritméticas y lógicas, permitiendo el uso de operadores, agrupación de expresiones, y variables o números como elementos básicos. A continuación, se explica de manera estructurada:

## 1. Regla Principal (prog)

- La regla prog es el **punto de entrada** de la gramática. Está diseñada para aceptar una o más expresiones (expr +), lo que significa que permite múltiples expresiones consecutivas.

## 2. Reglas de Expresión (expr)

- La regla expr define las diferentes formas de expresiones válidas en el lenguaje. Cada subregla dentro de expr tiene un nombre asociado (después del símbolo #) que facilita la identificación del tipo de expresión que representa. Estas subreglas cubren operaciones aritméticas, lógicas y de agrupación.
- **Agrupación con Paréntesis:**
  - expr: '(' expr ')' #PAR permite agrupar expresiones entre paréntesis, lo que ayuda a establecer prioridades entre operaciones.
- **Negación:**
  - '-' '(' expr ')' #NEGPARG y '-' atom #NEGATOM permiten aplicar la **negación** a una expresión entre paréntesis o a un solo elemento, como una variable o número.
  - '-' expr #NEG también define una **negación** general aplicada a cualquier expresión sin necesidad de paréntesis.
- **Operadores de Potencia y Aritméticos:**
  - expr '^' expr #POT representa la **potencia**, permitiendo elevar un número o variable a la potencia de otro.
  - expr ('\*' | '/') expr #MULDIV define las operaciones de **multiplicación** y **división**.

- `expr ('+' | '-') expr #SUMRES` permite las operaciones de **suma y resta**.
- **Operadores Lógicos:**
  - `expr '&&' expr #AND` y `expr '||' expr #OR` definen las operaciones **AND** y **OR** para trabajar con expresiones lógicas, evaluando si ambas (AND) o al menos una (OR) de las expresiones es verdadera.
- **Elemento Básico (atom):**
  - Cuando `expr` es simplemente un número o variable, se utiliza `atom`. Esta regla facilita la diferenciación entre una expresión compleja y un valor o variable individual.

### 3. Regla de Átomo (atom)

- La regla `atom` define los elementos básicos de una expresión, que pueden ser **variables** (ID) o **números** (NUMBER).
- **ID** representa una variable, que puede ser cualquier secuencia de letras (por ejemplo, `a`, `b`, `x`).
- **NUMBER** permite la inclusión de números enteros y decimales en las expresiones.

### 4. Definición de Tokens

- **ID y NUMBER:** Son tokens definidos para reconocer variables (ID) y números (NUMBER) en la gramática.
- **WS:** Representa los espacios en blanco (`[ \t\r\n]+`) y se omite en el análisis léxico gracias a `-> skip`, lo que permite que estos no afecten la estructura de la expresión.

```

1  grammar Examen;
2  prog:    expr '+';
3  expr:    '(' expr ')'          # PAR
4          | '-' '(' expr ')'    # NEGPAR
5          | expr '^' expr       # POT
6          | expr ('*' | '/') expr # MULDIV
7          | expr ('+' | '-') expr # SUMRES
8          | expr '&&' expr       # AND
9          | expr '||' expr      # OR
10         | '-' atom           # NEGATOM
11         | '-' expr           # NEG
12         | atom               # ATOM
13         ;
14  atom:    ID                  # VAR
15         |  NUMBER             # NUM
16         ;
17  ID      : [a-zA-Z]+ ;
18  NUMBER  : [0-9]+ ('.' [0-9]+)? ;
19  WS      : [ \t\r\n]+ -> skip ;

```

# explicación de las pruebas en cada parte

## 1. ExamenLexer.tokens

Este archivo es el resultado de la generación de tokens por parte de ANTLR. Para probar esta parte, primero definimos los tokens en la gramática para los operadores, números, identificadores y espacios en blanco (o WS) que hemos mencionado. Al ejecutar ANTLR, este archivo se generó con los códigos asignados a cada token (como vimos antes con ID=10, NUMBER=11, etc.).

### Pruebas realizadas:

- **Verificación de tokens:** Revisé que cada token estuviera correctamente identificado en el archivo. Esto implica que los símbolos +, -, &&, ||, entre otros, tengan su respectivo identificador numérico. De esta forma, confirmamos que el analizador léxico los reconoce adecuadamente y que la generación de tokens es precisa.
- **Pruebas de consistencia:** Comprobé que los códigos de tokens sean los mismos en el código y en este archivo, para evitar cualquier inconsistencia en el análisis posterior.

## 2. ExamenLexer.interp y Examen.interp

Ambos archivos contienen una interpretación de los tokens según el código generado. **ExamenLexer.interp** muestra cómo se interpretan los tokens generados

por el lexer, mientras que **Examen.interp** refleja el análisis sintáctico de las expresiones según la gramática.

#### Pruebas realizadas:

- **Prueba de reconocimiento léxico:** En ExamenLexer.interp, probé varias expresiones simples como  $a + b$ ,  $3 * 4$ , o  $x \&\& y \parallel z$ . Esto ayuda a comprobar que cada símbolo sea interpretado como el token adecuado.
- **Prueba de análisis sintáctico:** En Examen.interp, comprobé cómo la gramática reconoce y organiza cada expresión. Probé expresiones más complejas con paréntesis y negaciones para asegurarme de que la estructura del árbol sintáctico reflejara correctamente la precedencia y el agrupamiento definidos en la gramática.
- **Prueba de operadores lógicos:** También hice pruebas para asegurarnos de que los operadores  $\&\&$  y  $\parallel$  se procesaran correctamente dentro de expresiones complejas.

### 3. ExamenVisitor.py

Este archivo contiene el visitor que recorre el árbol sintáctico y evalúa las expresiones aritméticas y lógicas. Aquí es donde se implementan las reglas de cómo procesar y devolver resultados para cada tipo de expresión que define la gramática.

#### Pruebas realizadas:

- **Evaluación aritmética:** Primero, realicé pruebas con operaciones aritméticas básicas (+, -, \*, /) y comprobé que el visitor devolviera el resultado correcto. Ejemplos de pruebas incluyen expresiones como  $3 + 5$ ,  $10 - 2$ , y  $7 * 4 / 2$ .
- **Evaluación de potencias:** Probé la operación de potencia (^) para asegurarme de que el visitor realizara correctamente esta operación, incluyendo expresiones como  $2 ^ 3$  y  $4 ^ 2$ .
- **Operadores lógicos:** Realicé pruebas con expresiones lógicas que incluían  $\&\&$  y  $\parallel$ . Probé casos como  $true \&\& false$ ,  $false \parallel true$ , y combinaciones de estas expresiones junto con aritmética para verificar que el visitor manejara correctamente tanto el retorno booleano como la evaluación de las condiciones.
- **Manejo de variables y errores:** También hice pruebas con expresiones que contenían variables indefinidas para ver si el visitor generaba un error adecuado. Esto incluyó casos donde una variable era llamada sin haber sido definida, lo cual verifica el manejo de errores en el visitor.

#### 4. main.py

Este archivo parece ser el archivo principal que ejecuta la gramática y el visitor, permitiendo la prueba general de todo el flujo desde el análisis léxico hasta la evaluación de las expresiones.

#### Pruebas realizadas:

- **Integración de componentes:** Comprobé que el archivo main.py pudiera procesar una expresión desde el lexer hasta la evaluación final con el visitor. Esto incluyó pasar expresiones variadas como entrada y verificar que el visitor produjera el resultado correcto.
- **Prueba de expresiones combinadas:** Probé expresiones combinadas de operadores lógicos y aritméticos como  $a + b * c \&\& d || e$ . Esto fue útil para confirmar que main.py manejara correctamente el flujo y priorización de operadores.

#### 5. Pruebas

En expresiones aritméticas si es correcta imprime el árbol de sintaxis y hace la evaluación correctamente como se muestra en las capturas

```
Evaluando expresión: (3 + 5) * 2
Árbol de sintaxis para '(3 + 5) * 2':
(prog (expr (expr ( (expr (expr (atom 3)) + (expr (atom 5)))) ) * (expr (atom 2))))
Resultado: [16.0]

Evaluando expresión: 4 + (7 - 2) / 3
Árbol de sintaxis para '4 + (7 - 2) / 3':
(prog (expr (expr (atom 4)) + (expr (expr ( (expr (expr (atom 7)) - (expr (atom 2))) ) / (expr (atom 3)))))
Resultado: [5.666666666666667]
```

En expresiones con variables tanto aritméticas como lógicas solo imprime el árbol de sintaxis con ningún error

```
Evaluando expresión: x && y || z
Árbol de sintaxis para 'x && y || z':
(prog (expr (expr (expr (atom x)) && (expr (atom y))) || (expr (atom z))))

Evaluando expresión: x * (y / z + 3) - 9
Árbol de sintaxis para 'x * (y / z + 3) - 9':
(prog (expr (expr (expr (atom x)) * (expr ( (expr (expr (expr (atom y)) / (expr (atom z))) + (expr (atom 3))) ))) - (expr (atom 9))))

Evaluando expresión: (-x ^ 2) * 3
Árbol de sintaxis para '(-x ^ 2) * 3':
(prog (expr (expr ( (expr (expr - (atom x)) ^ (expr (atom 2))) ) * (expr (atom 3)))))
```

En caso de operaciones lógicas se evaluaron como las compuertas lógicas

```
Evaluando expresión: 1 && 1
Árbol de sintaxis para '1 && 1':
(prog (expr (expr (atom 1)) && (expr (atom 1))))
Resultado: [1.0]

Evaluando expresión: 0 && 0
Árbol de sintaxis para '0 && 0':
(prog (expr (expr (atom 0)) && (expr (atom 0))))
Resultado: [0.0]

Evaluando expresión: 1 || 0
Árbol de sintaxis para '1 || 0':
(prog (expr (expr (atom 1)) || (expr (atom 0))))
Resultado: [1.0]
```

En caso de operaciones lógicas y aritméticas se hicieron las siguientes pruebas, ya que en este tipo de pruebas Python responde de manera diferente a otro lenguaje , pues evalúa y toma como correcta el último dato con el que se quedo, dado esto, se considera que son correctas las pruebas

```
Evaluando expresión: -((3 || 5) && (0 && 1)) || 4
Árbol de sintaxis para '-((3 || 5) && (0 && 1)) || 4':
(prog (expr (expr - ( (expr (expr ( (expr (expr (atom 3)) || (expr (atom 5))) ) && (expr ( (expr (expr (atom 0)) && (expr (atom 1))) ))) ) || (expr (atom 4)))))
Resultado: [4,0]

Evaluando expresión: -(10 || (6 && 2)) && (1 + 1) || 8
Árbol de sintaxis para '-(10 || (6 && 2)) && (1 + 1) || 8':
(prog (expr (expr - ( (expr (expr (atom 10)) || (expr ( (expr (expr (atom 6)) && (expr (atom 2))) ))) ) && (expr ( (expr (expr (atom 1)) + (expr (atom 1)) ) ))) || (expr (atom 8)))))
Resultado: [2,0]

Evaluando expresión: -((5 && 0) || (4 || 1)) && (3 + 2)
Árbol de sintaxis para '-((5 && 0) || (4 || 1)) && (3 + 2)':
(prog (expr (expr - ( (expr (expr ( (expr (expr (atom 5)) && (expr (atom 0))) ) || (expr ( (expr (expr (atom 4)) || (expr (atom 1))) ))) ) && (expr ( (expr (expr (atom 3)) + (expr (atom 2))) )))))
Resultado: [5,0]
```

Para operaciones aritméticas mas complejas y con potencia quedaron así

```
Evaluando expresión: -((5 ^ 2) + (3 * 4) - (8 / 2)) + (2 ^ 3)
Árbol de sintaxis para '-((5 ^ 2) + (3 * 4) - (8 / 2)) + (2 ^ 3)':
(prog (expr (expr - ( (expr (expr (expr ( (expr (expr (atom 5)) ^ (expr (atom 2))) ) + (expr ( (expr (expr (atom 3)) * (expr (atom 4))) ))) - (expr ( (expr (expr (atom 8)) / (expr (atom 2))) ))) ) + (expr ( (expr (expr (atom 2)) ^ (expr (atom 3))) )))
Resultado: [-25,0]

Evaluando expresión: (3 * (2 + 5)) - (4 ^ 2) + (12 / 3) * -1
Árbol de sintaxis para '(3 * (2 + 5)) - (4 ^ 2) + (12 / 3) * -1':
(prog (expr (expr (expr ( (expr (expr (atom 3)) * (expr ( (expr (expr (atom 2)) + (expr (atom 5))) ))) ) - (expr ( (expr (expr (atom 4)) ^ (expr (atom 2))) ))) + (expr (expr ( (expr (expr (atom 12)) / (expr (atom 3))) ) * (expr - (atom 1))))))
Resultado: [1,0]

Evaluando expresión: -((6 / 2) * (3 + 5) - (2 + 4)) + 10
Árbol de sintaxis para '-((6 / 2) * (3 + 5) - (2 + 4)) + 10':
(prog (expr (expr - ( (expr (expr (expr ( (expr (expr (atom 6)) / (expr (atom 2))) ) * (expr ( (expr (expr (atom 3)) + (expr (atom 5))) ))) - (expr ( (expr (expr (atom 2)) + (expr (atom 4))) ))) ) + (expr (atom 10)))))
Resultado: [-8,0]
```

En caso de una mala sintaxis el programa imprimirá lo siguiente

```
Evaluando expresión: -((5 ** 2) + (3 * 4) - (8 / 2)) + (2 ** 3)
line 1:6 no viable alternative at input '-((5**'
line 1:6 extraneous input '**' expecting {'(', '-', ID, NUMBER}
line 1:38 extraneous input '**' expecting {'(', '-', ID, NUMBER}
Árbol de sintaxis para '-((5 ** 2) + (3 * 4) - (8 / 2)) + (2 ** 3)':
(prog (expr (expr - ( (expr (expr (expr ( (expr (expr (atom 5)) * (expr * (atom 2))) ) + (expr ( (expr (expr (atom 3)) * (expr (atom 4))) ))) - (expr ( (expr (expr (atom 8)) / (expr (atom 2))) ))) ) + (expr ( (expr (expr (atom 2)) * (expr * (atom 3))) )))
Resultado: [None, None, 24,0]
```

En caso de división por 0 dara el error.

```
Evaluando expresión: (2 * 5 + 10) / 0
Árbol de sintaxis para '(2 * 5 + 10) / 0':
(prog (expr (expr ( (expr (expr (expr (atom 2)) * (expr (atom 5))) + (expr (atom 10))) ) ) / (expr (atom 0)))))
Error: División por cero!
Error: División por cero!
Resultado: [None]
```

En caso de paréntesis no balanceados invocara la función para evaluar paréntesis e imprimirá en caso de error

```
Evaluando expresión: -((3 || 5 && (0 && 1)) || 4
No esta balanceada

Evaluando expresión: -(10 || (6 && 2)) && 1 + 1) || 8
No esta balanceada

Evaluando expresión: -((5 && 0) || (4 || 1) && (3 + 2)
No esta balanceada
```